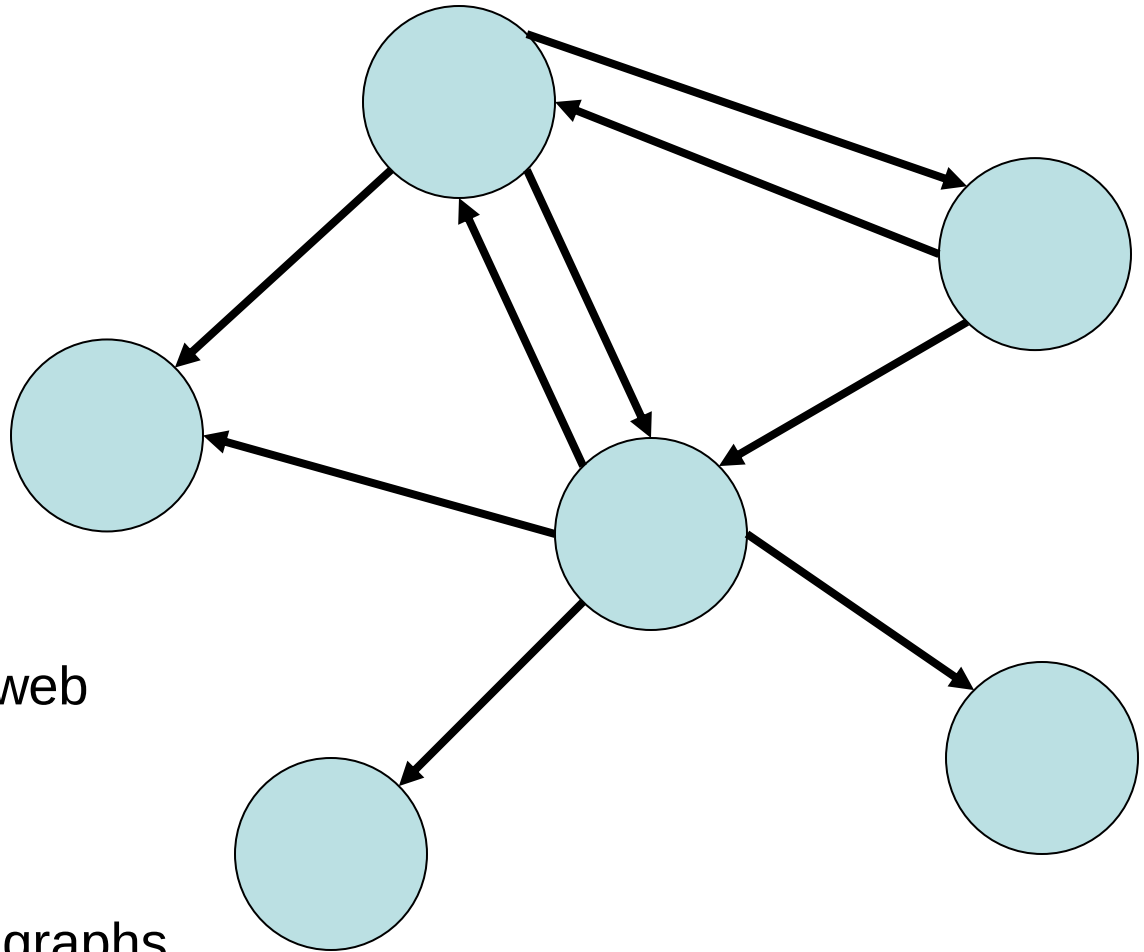


Design and Analysis of Algorithm

Topic: Graphs

Graphs

A graph consists of a set of **nodes** connected by **edges**.

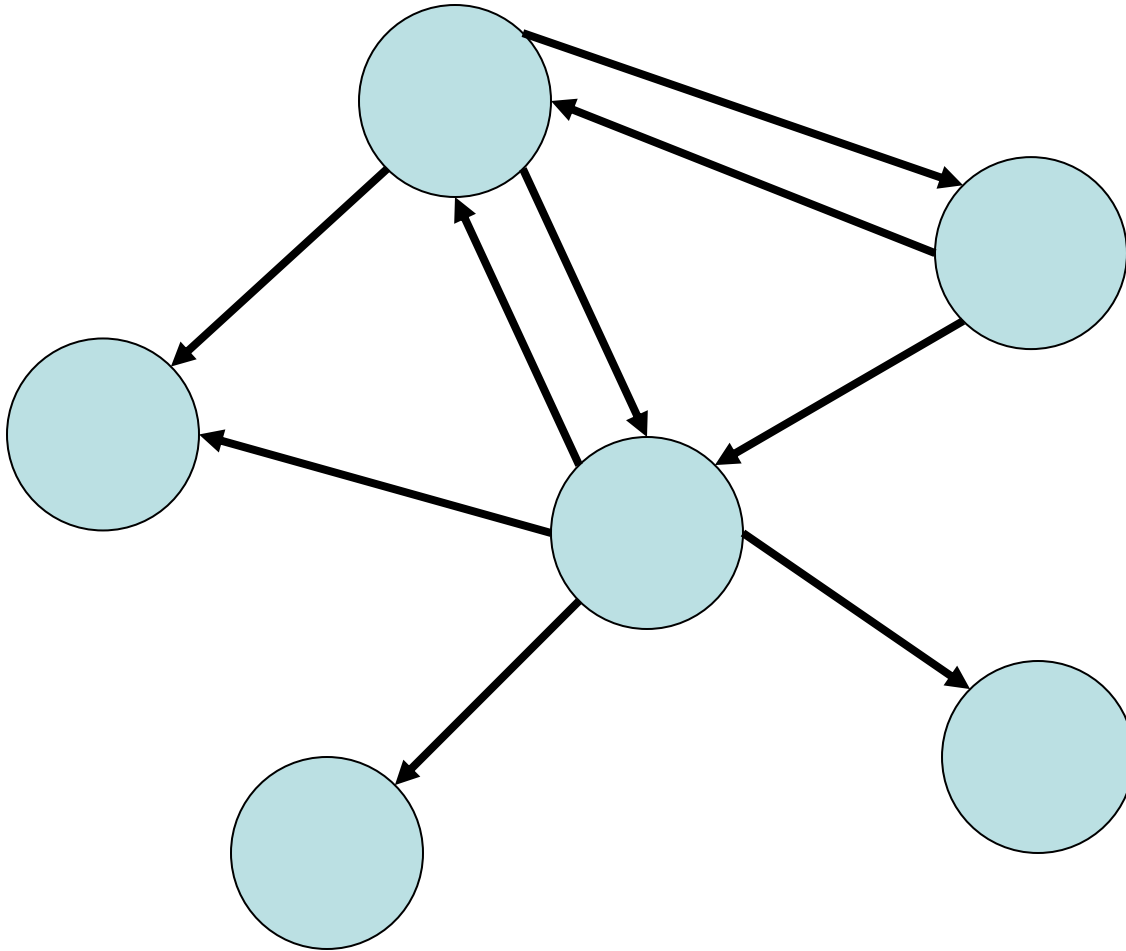


Graphs can model:

- Sites and links on the web
- Disease outbreaks
- Social networks
- Geographies
- Task and dependency graphs
- and more...

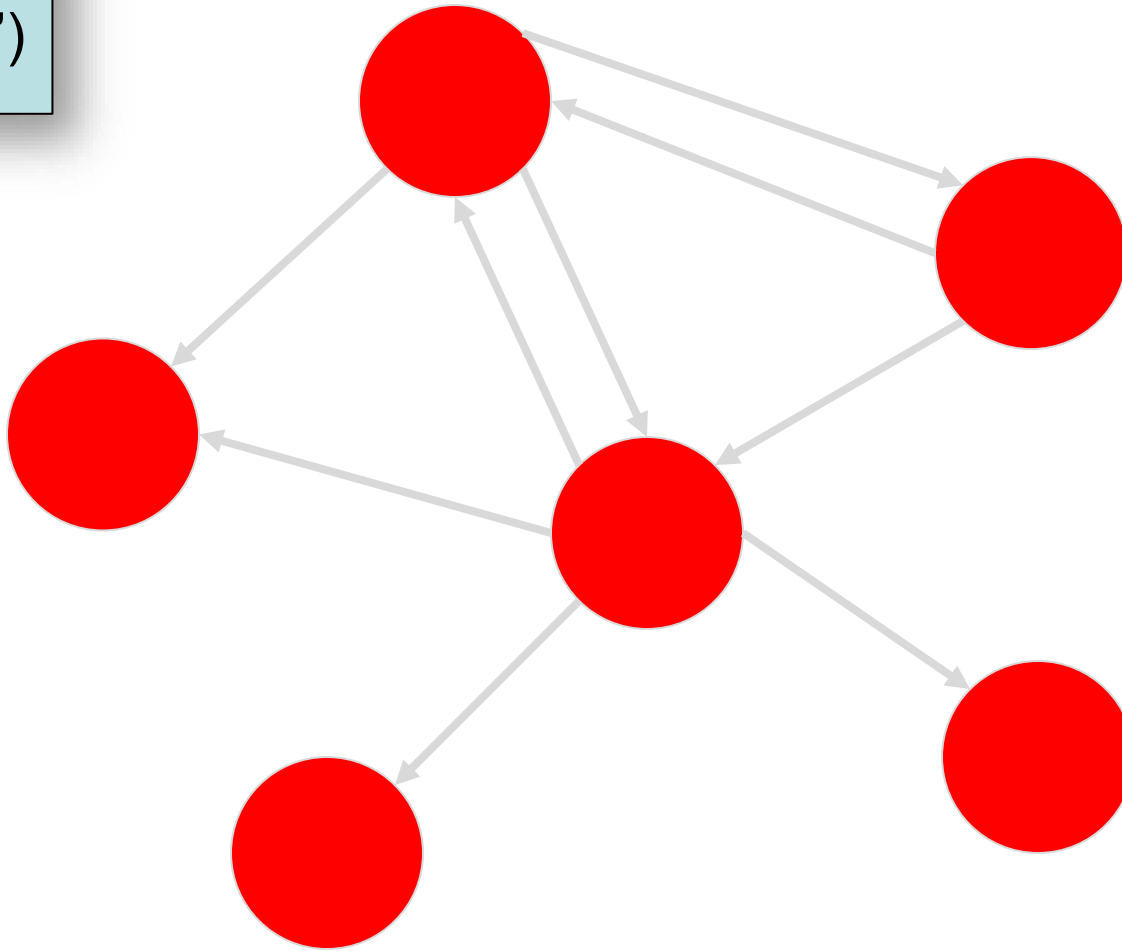
Graphs

A graph consists of a set of **nodes** connected by **edges**.



Nodes

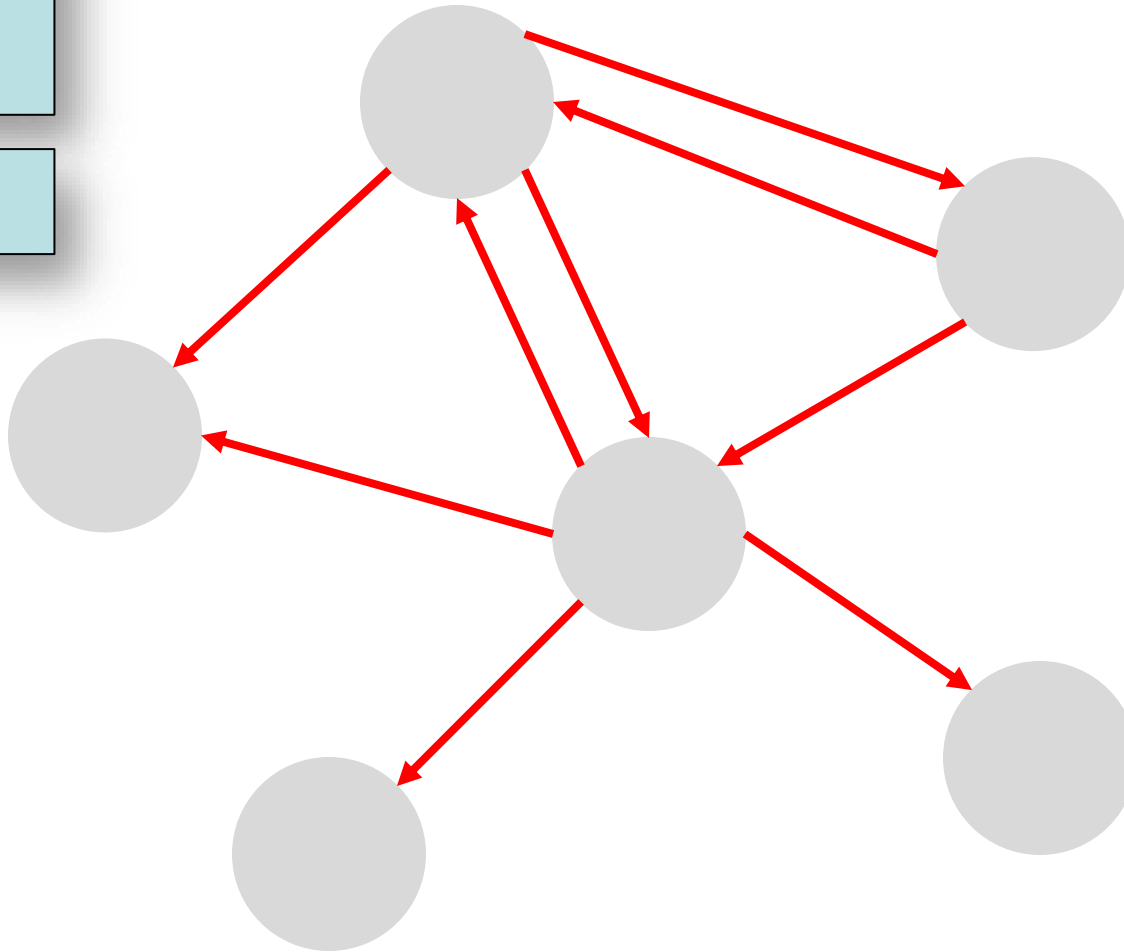
Nodes
("Vertices")



Edges

Edges
("Arcs")

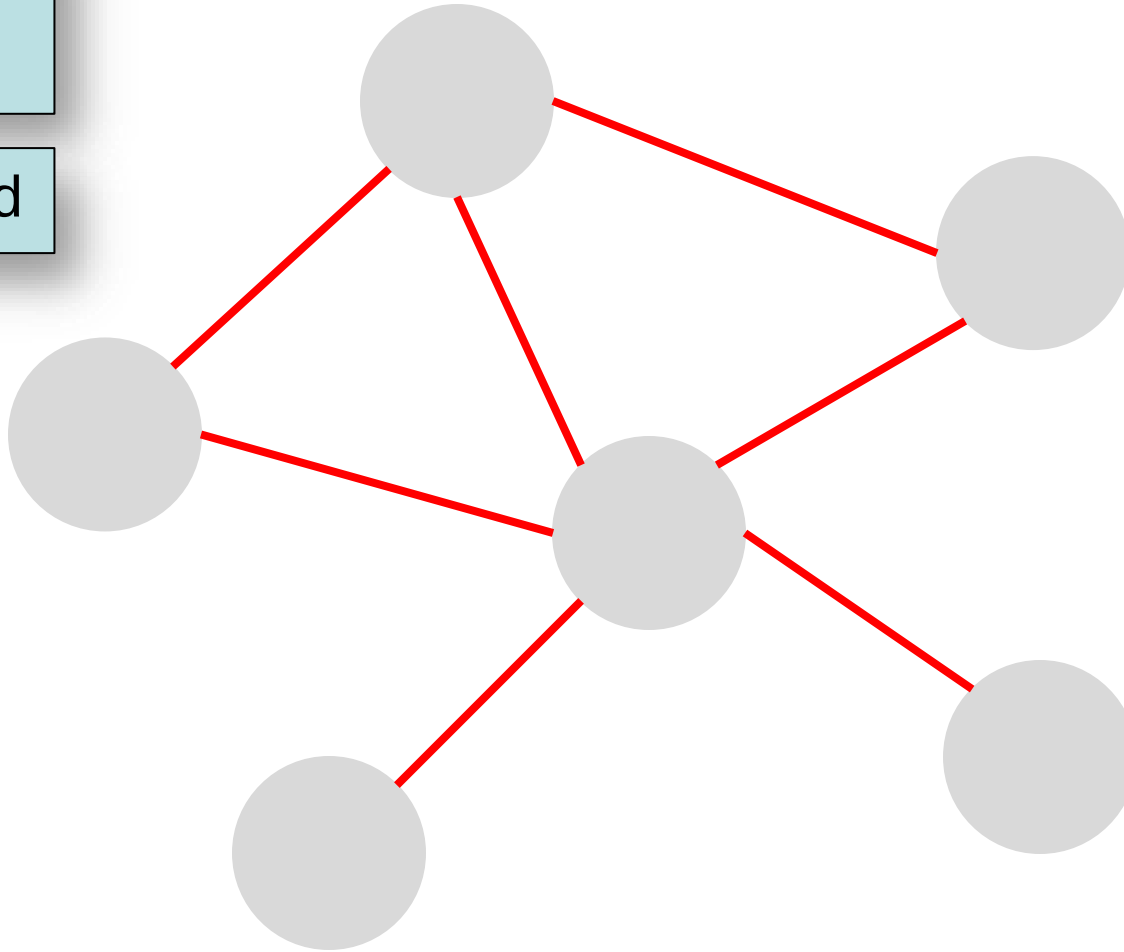
Directed



Edges

Edges
("Arcs")

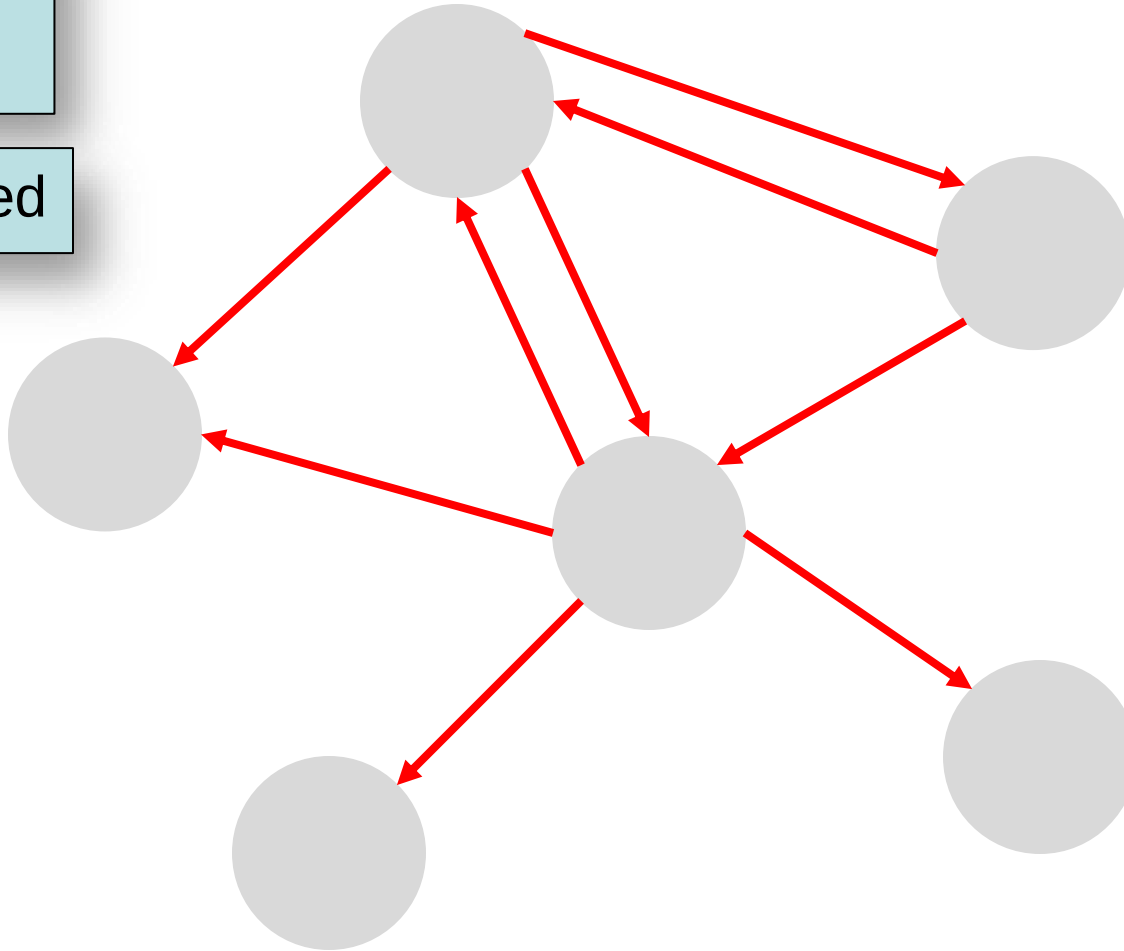
Undirected



Edges

Edges
("Arcs")

Unweighted

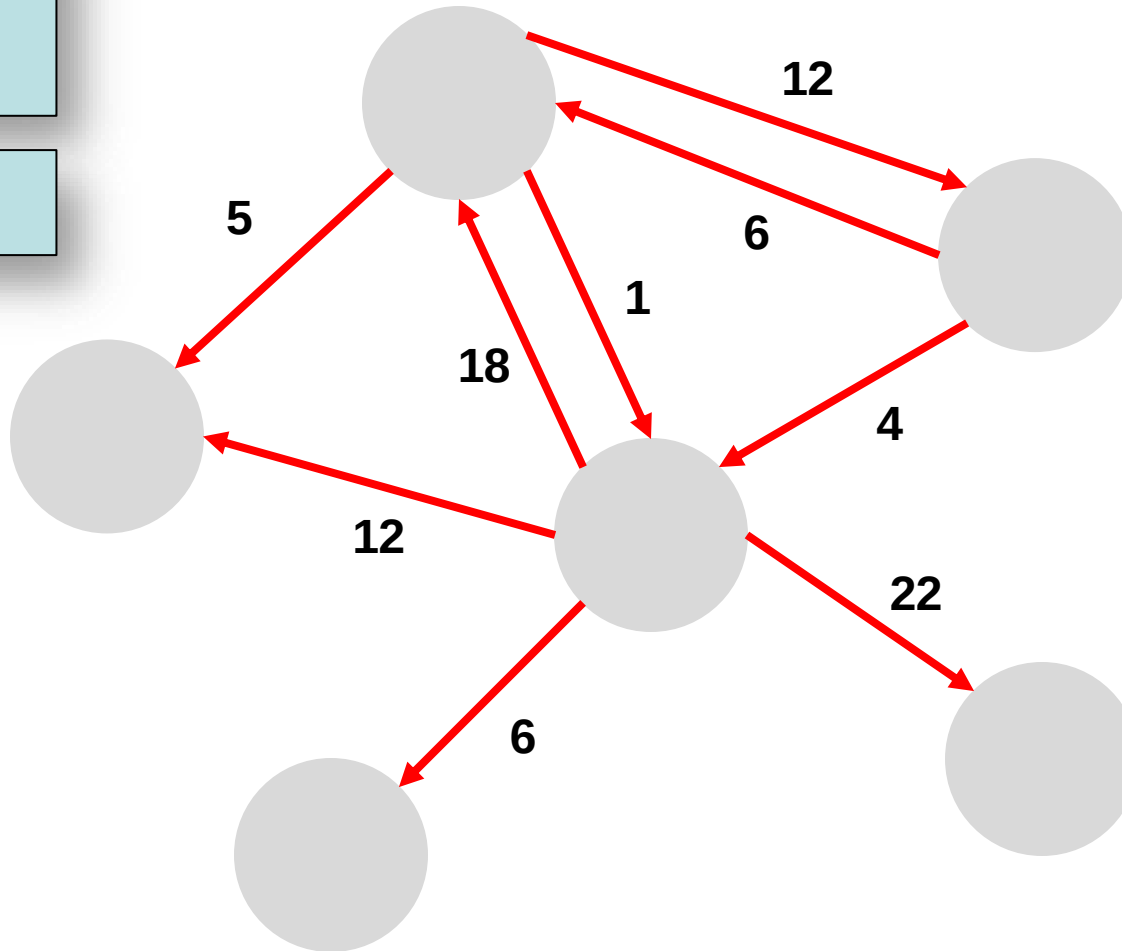


Weights

A weight is a cost associated with a given edge.

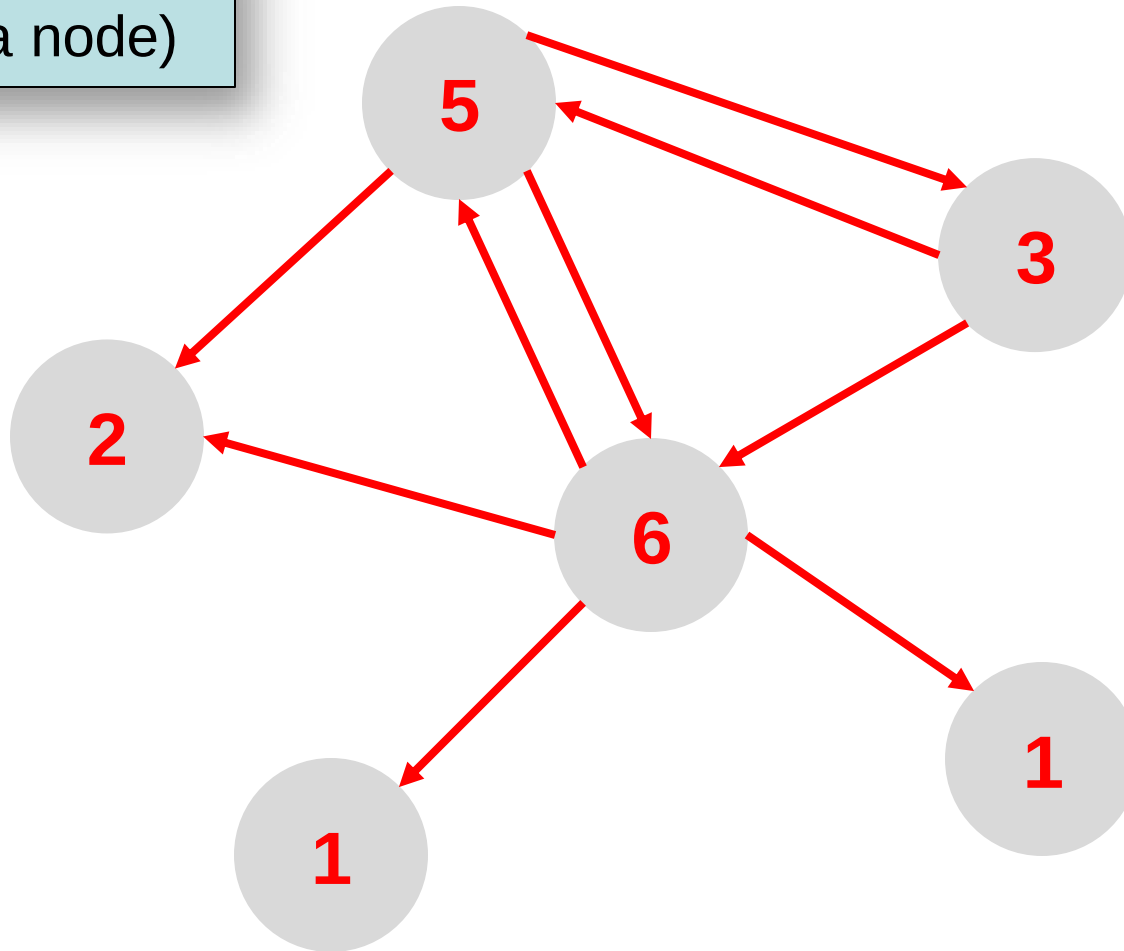
Edges
("Arcs")

Weighted



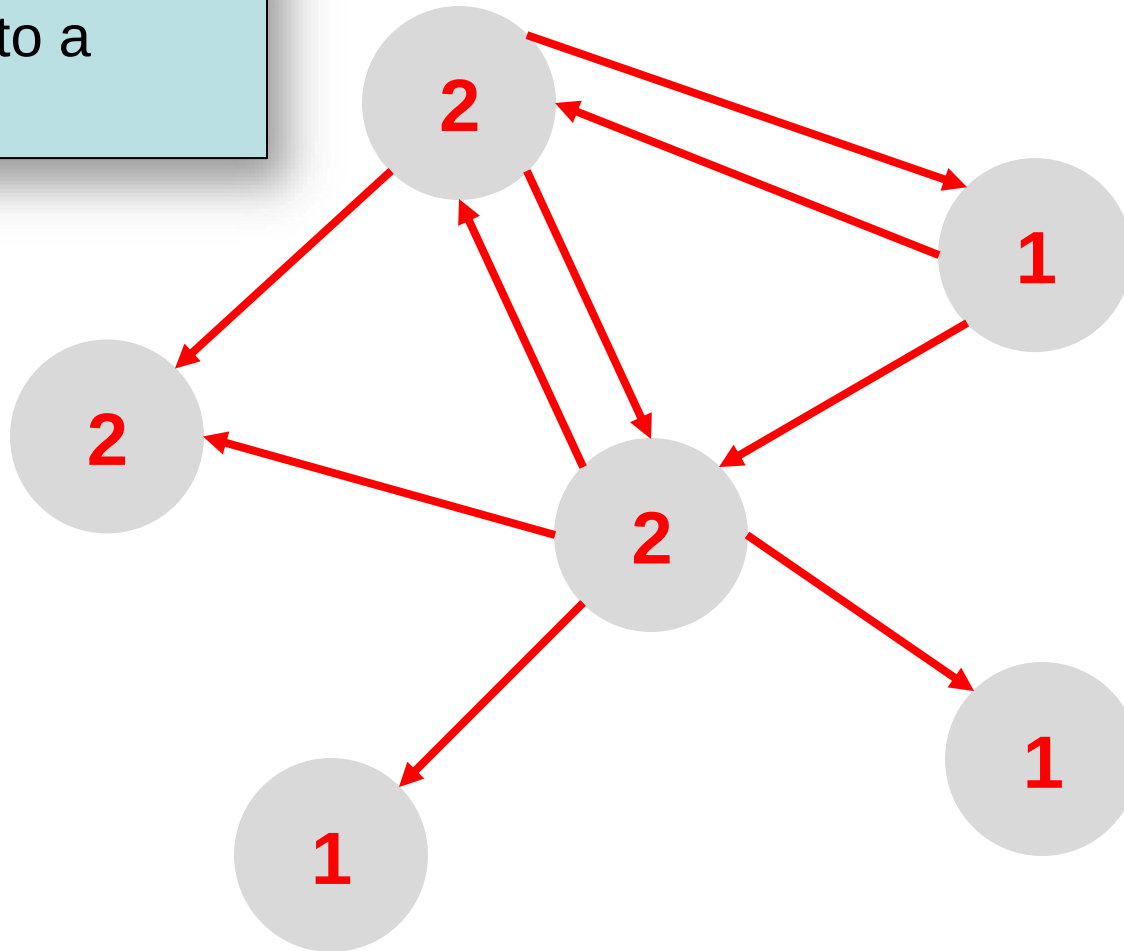
Degree

Degree (# edges touching a node)



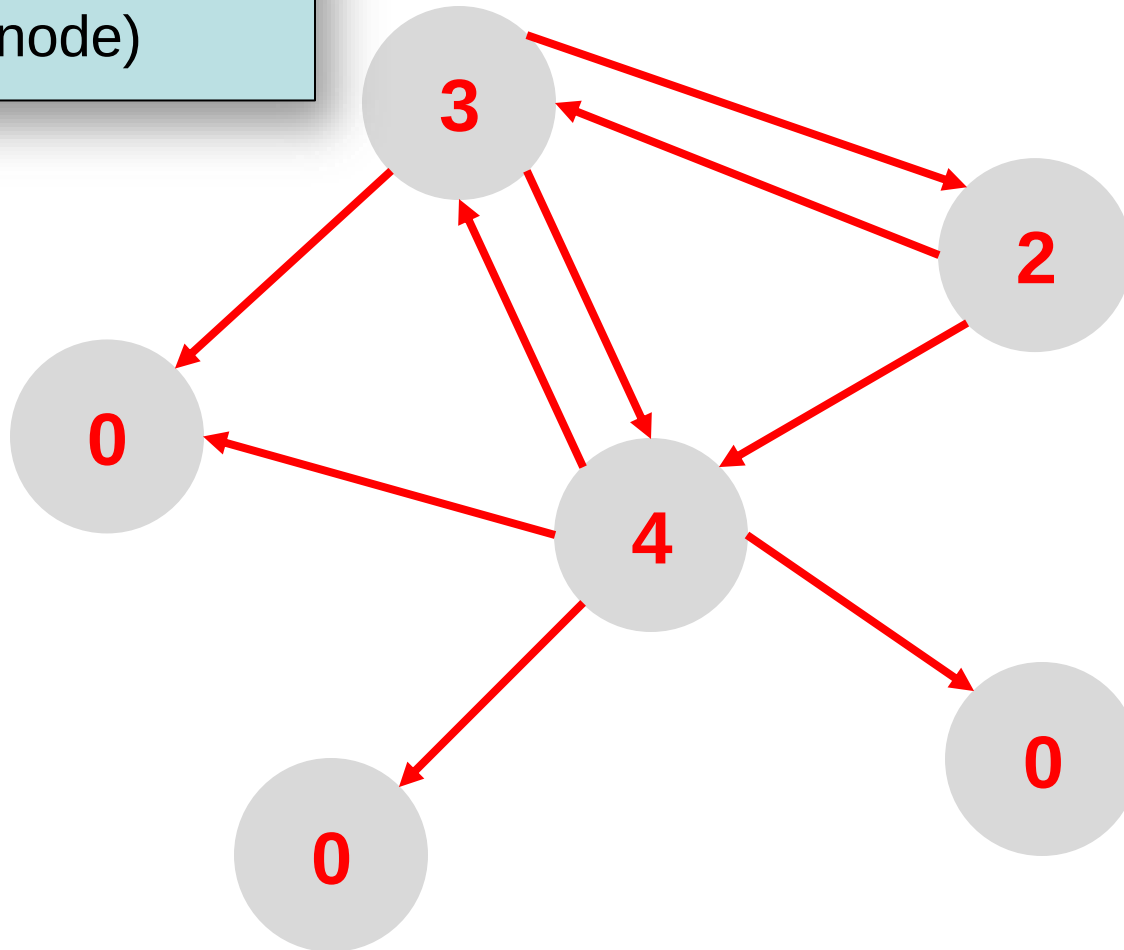
Degree

In-degree (# edges coming into a node)



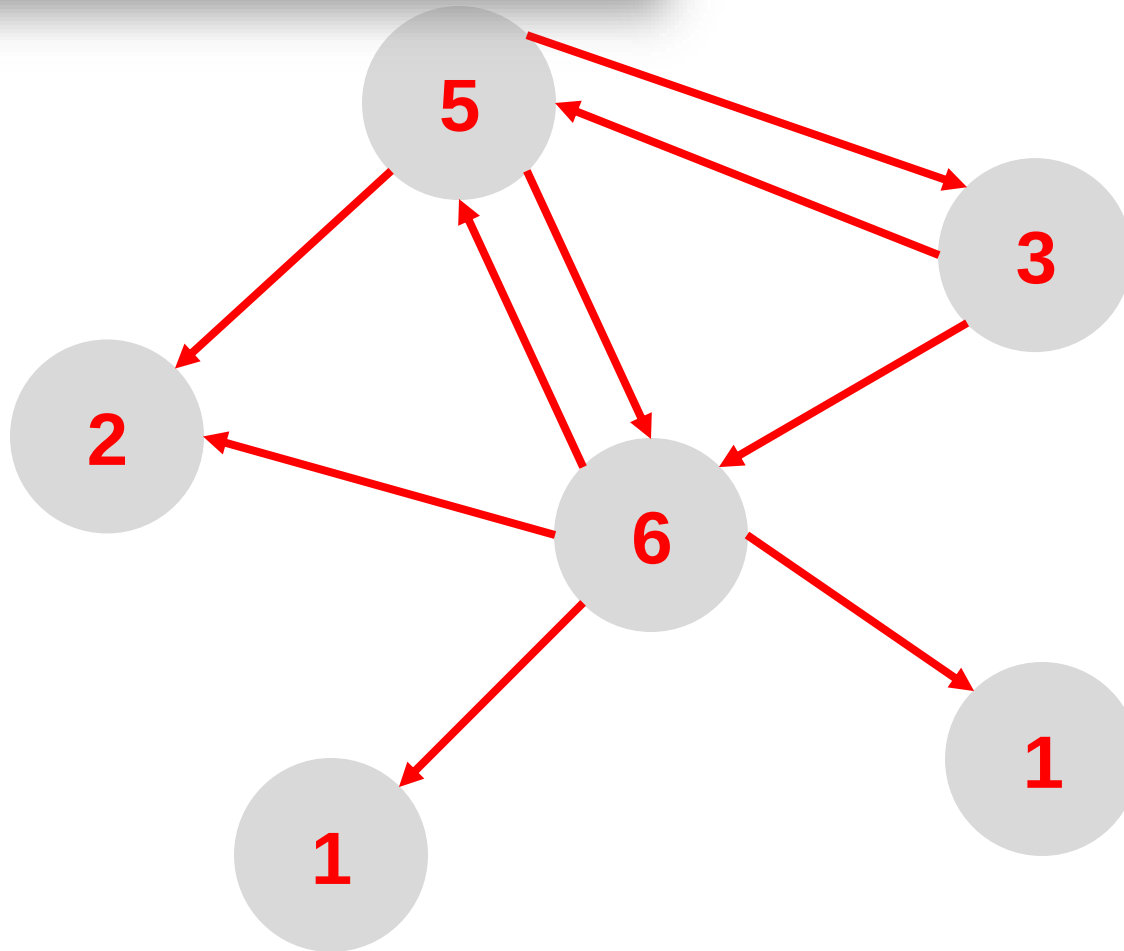
Degree

Out-degree (# edges leaving a node)



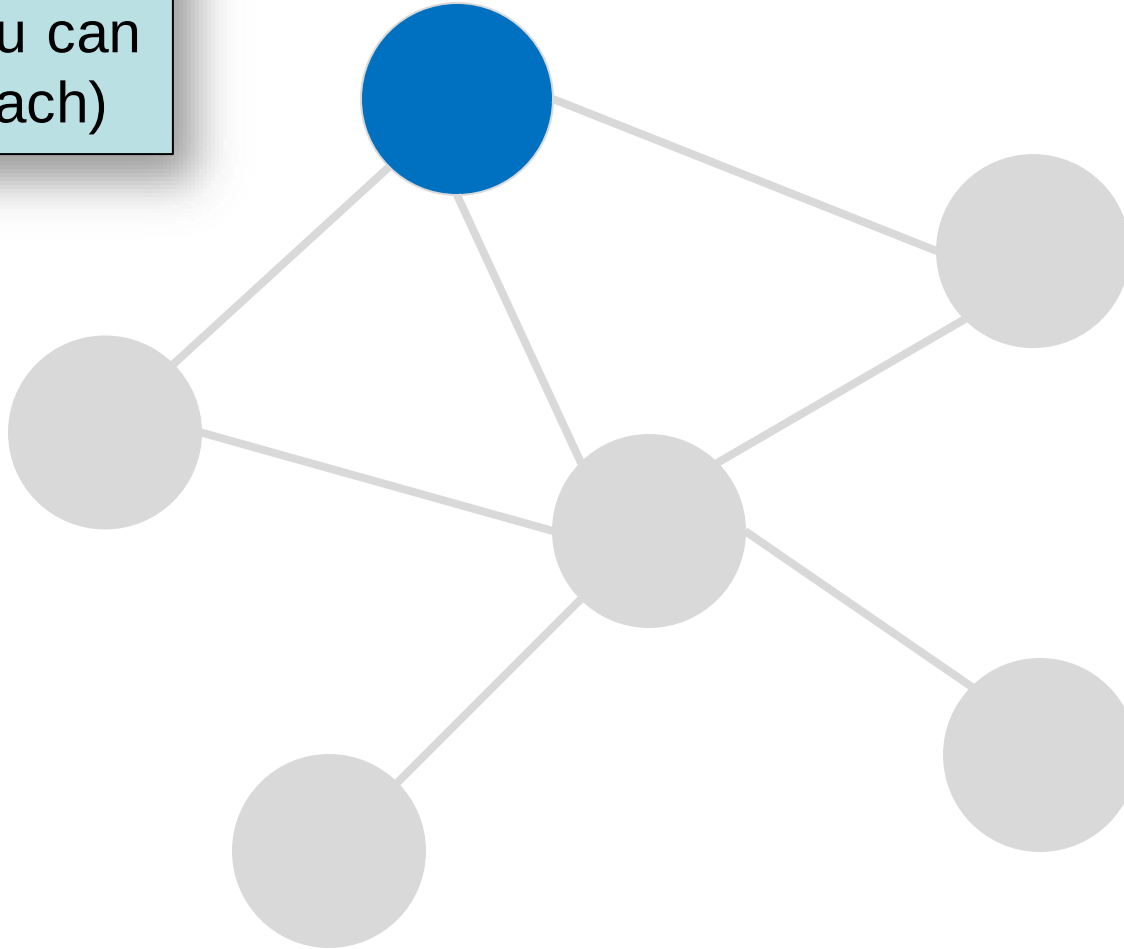
Degree

Degree = In-degree + Out-degree



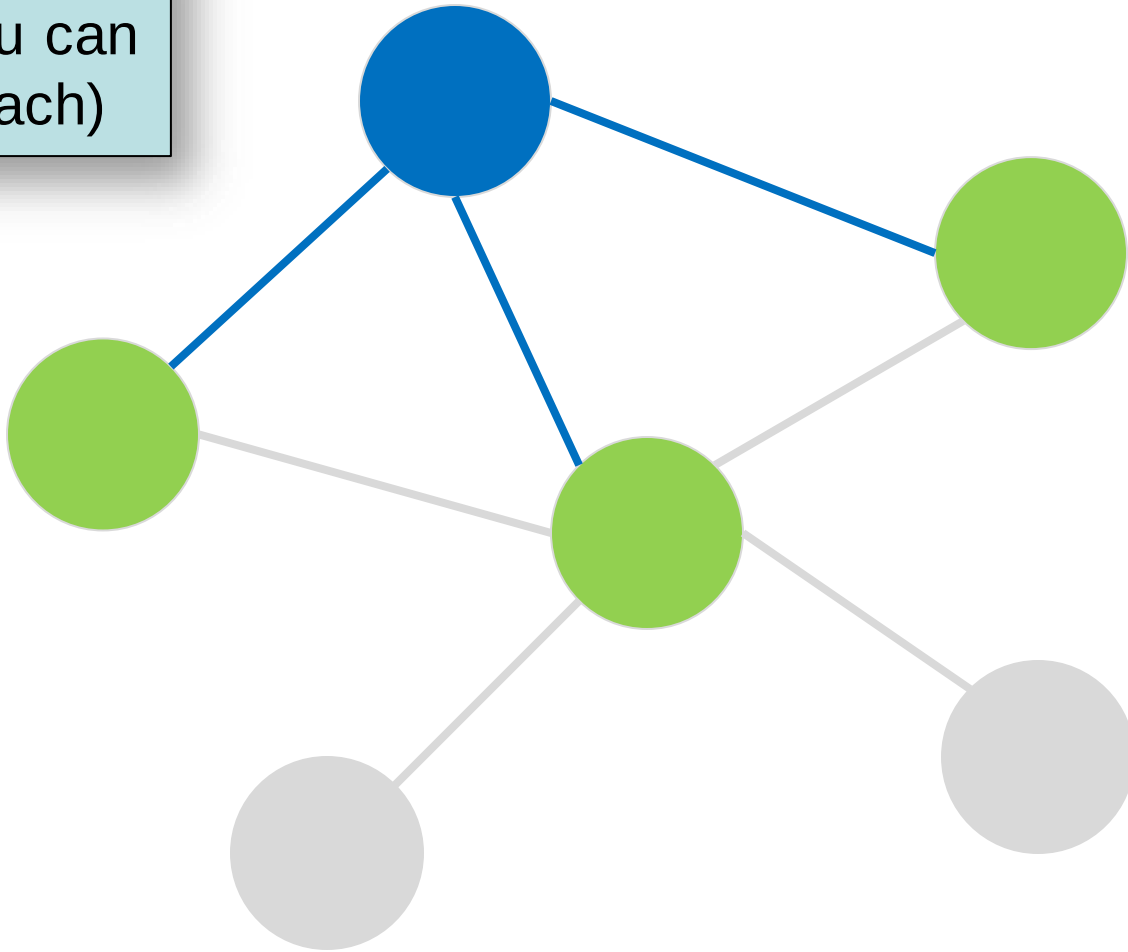
Neighbors

Neighbors
(nodes you can
directly reach)



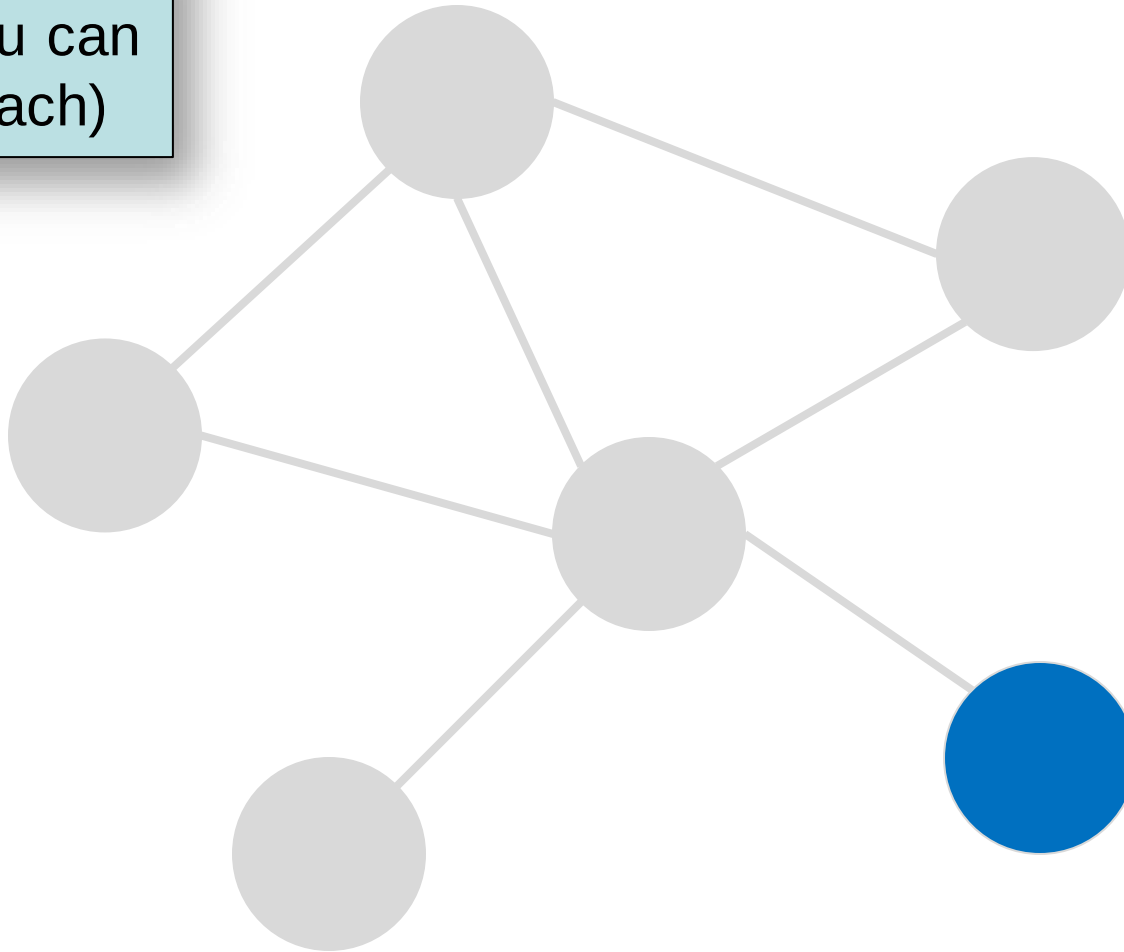
Neighbors

Neighbors
(nodes you can
directly reach)



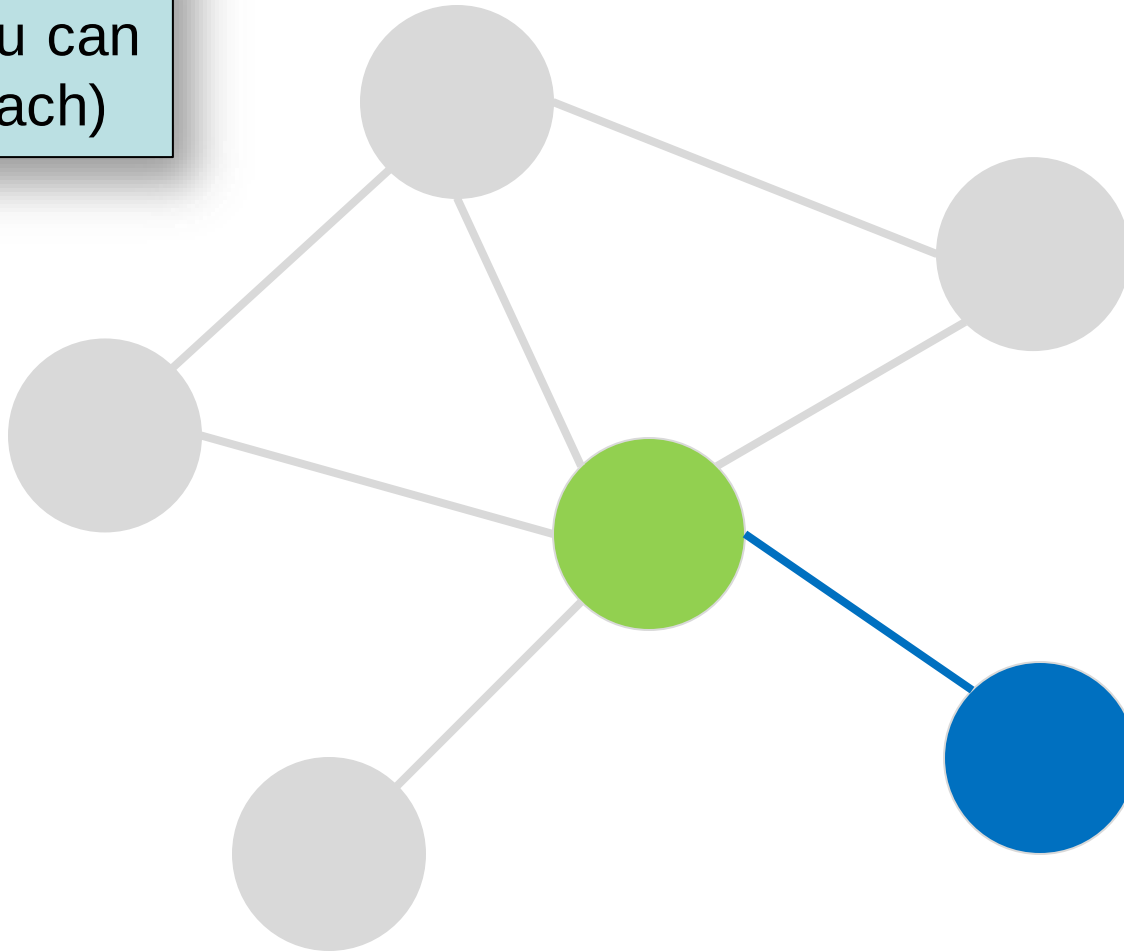
Neighbors

Neighbors
(nodes you can
directly reach)



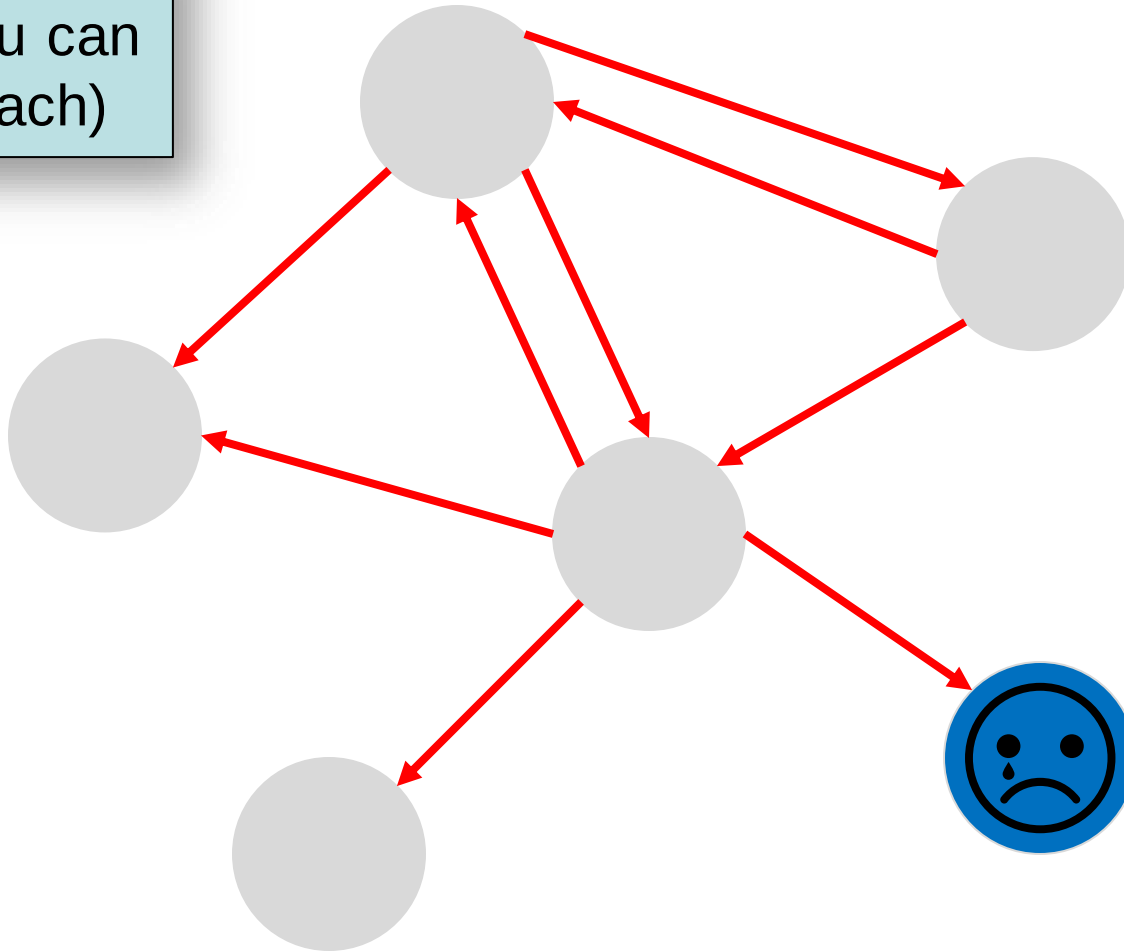
Neighbors

Neighbors
(nodes you can
directly reach)



Neighbors

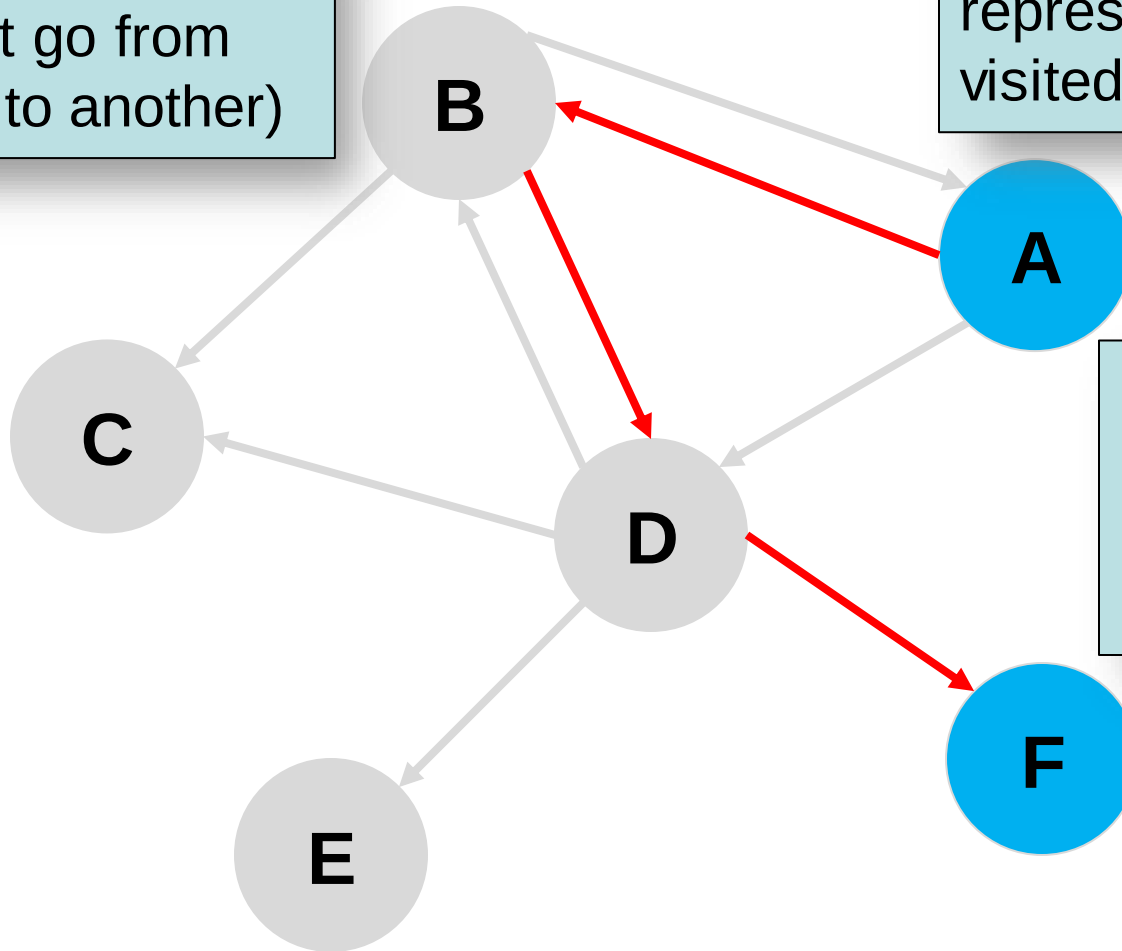
Neighbors
(nodes you can
directly reach)



Paths

Path (sequence of edges that go from one node to another)

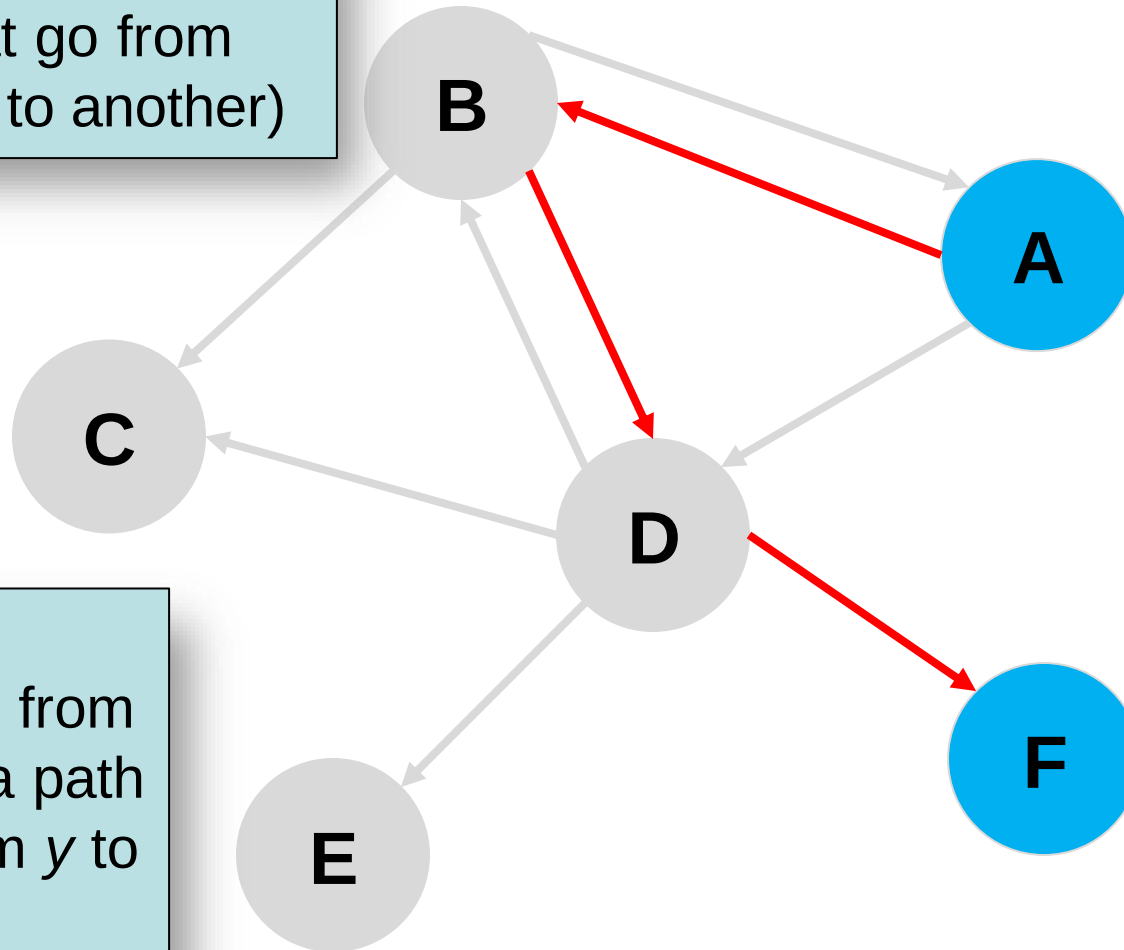
Paths are represented as nodes visited or edges taken



Path length = 3
(number of nodes or edges in the path)

Paths

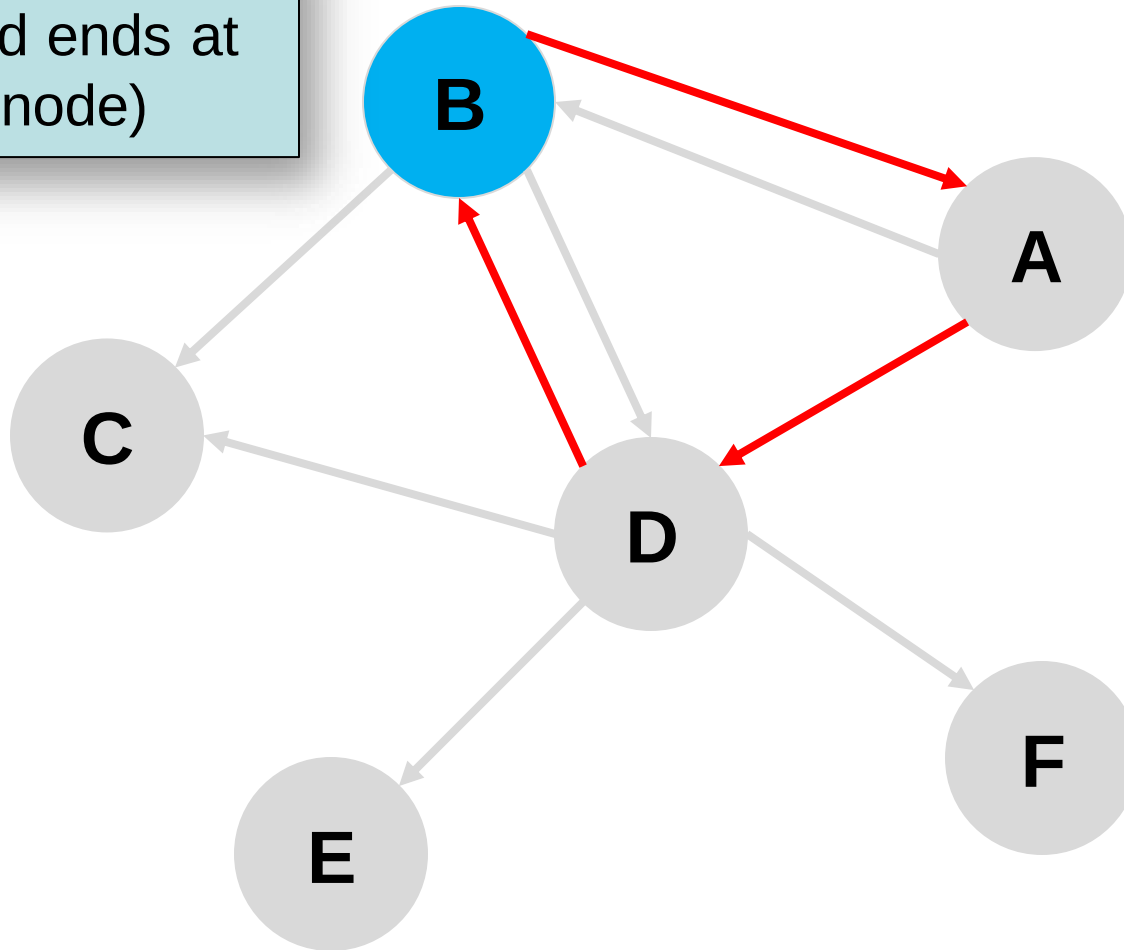
Path (sequence of edges that go from one node to another)



Node x is reachable from node y if a path exists from y to x .

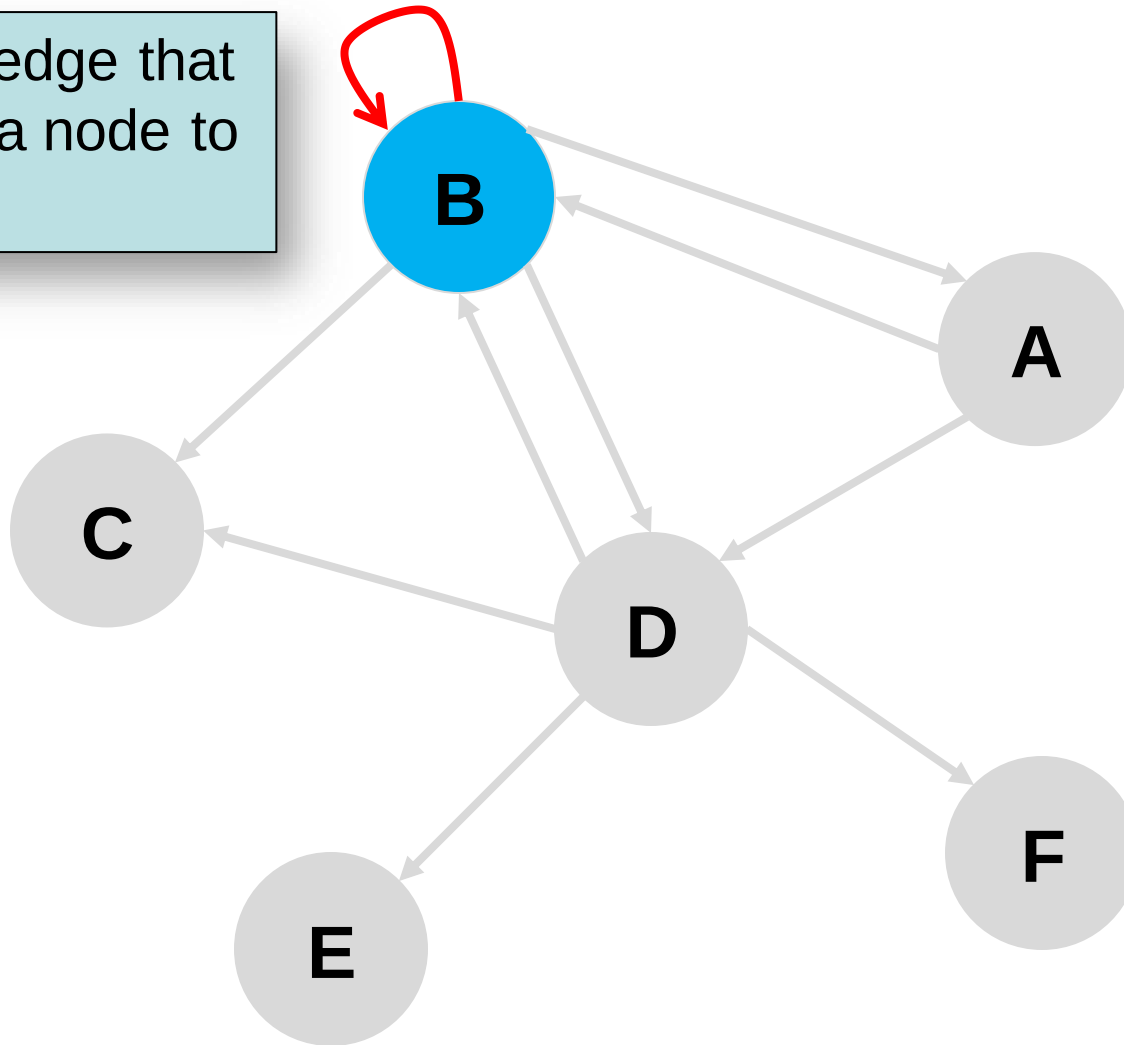
Cycles

Cycle (path that begins and ends at the same node)



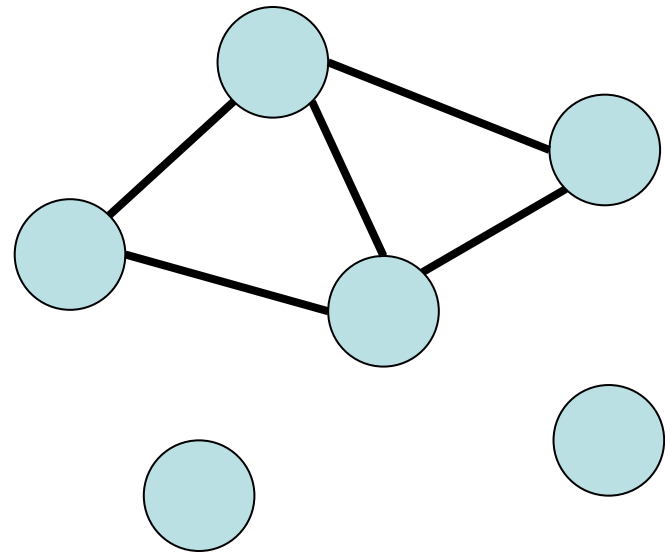
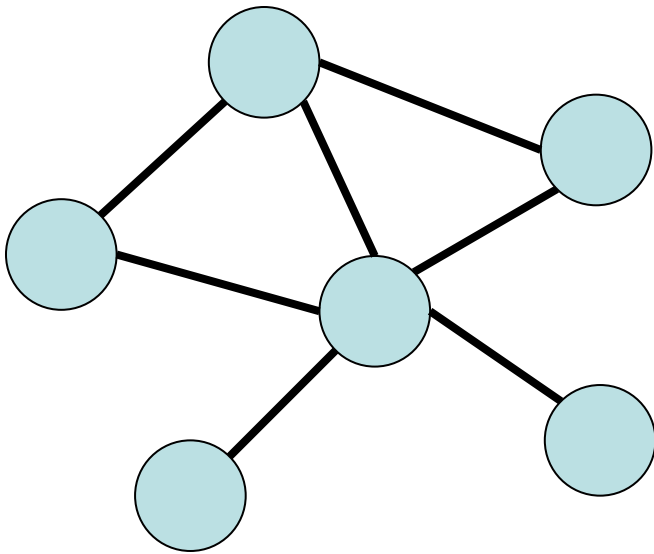
Loops

Loop (an edge that connects a node to itself)



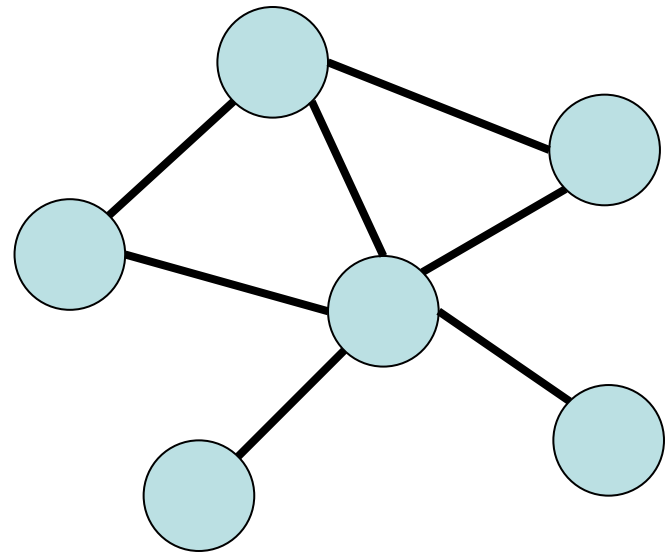
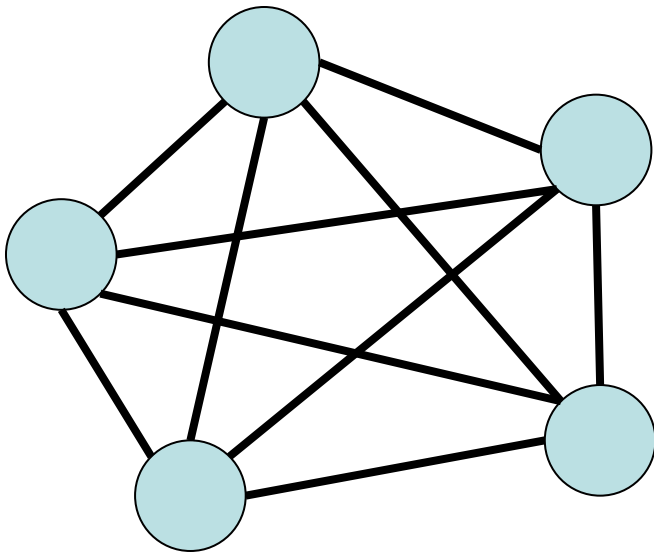
Graph Properties

A graph is **connected** if every node is reachable from every other node.



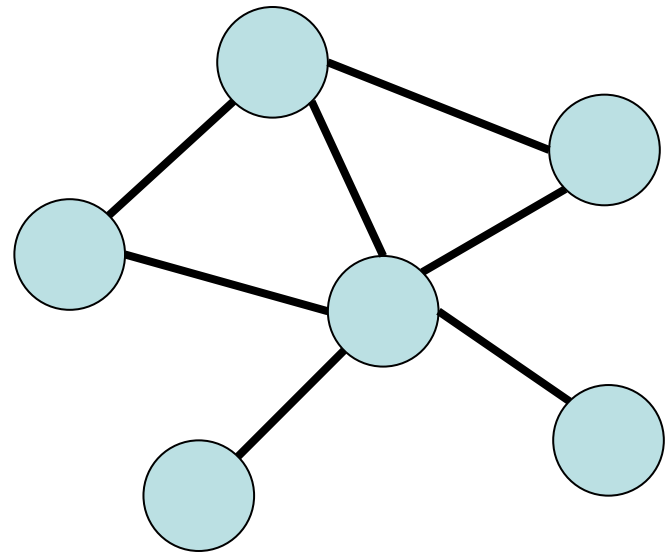
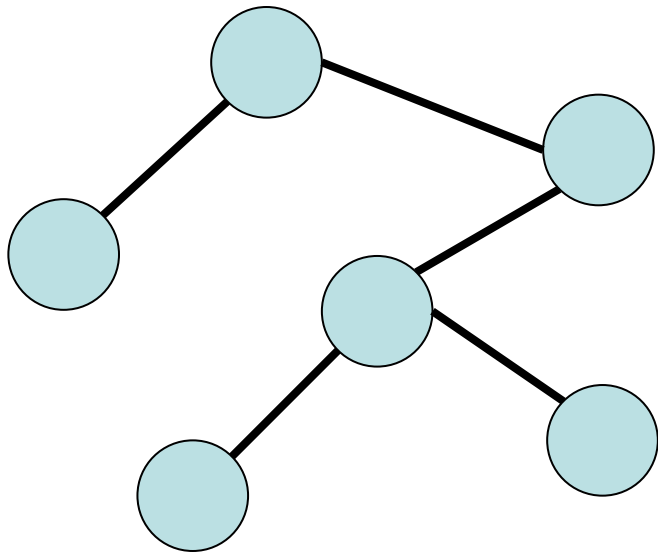
Graph Properties

A graph is **complete** if every node has a direct edge to every other node.



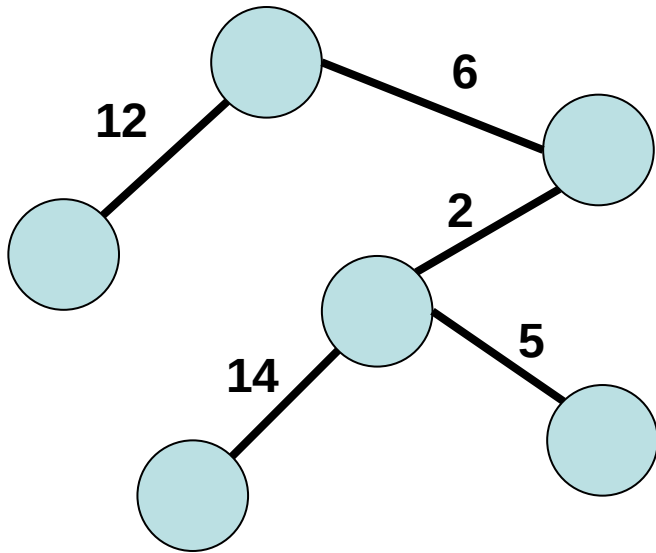
Graph Properties

A graph is **acyclic** if it does not contain any cycles.

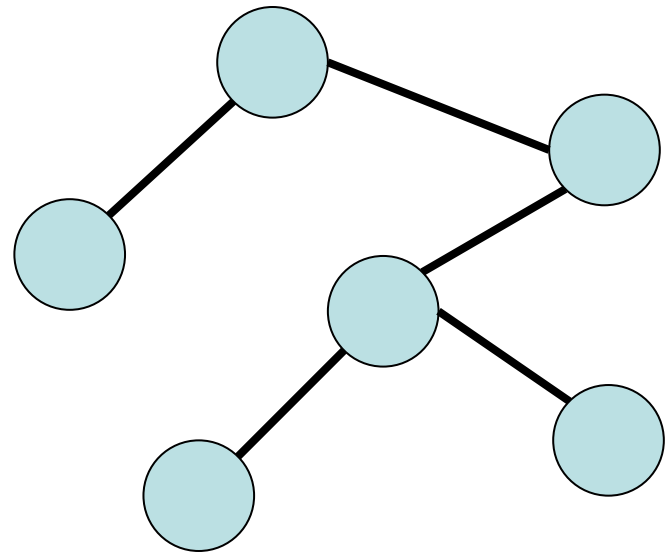


Graph Properties

A graph is **weighted** if its edges have weights, or **unweighted** if its edges do not have weights.



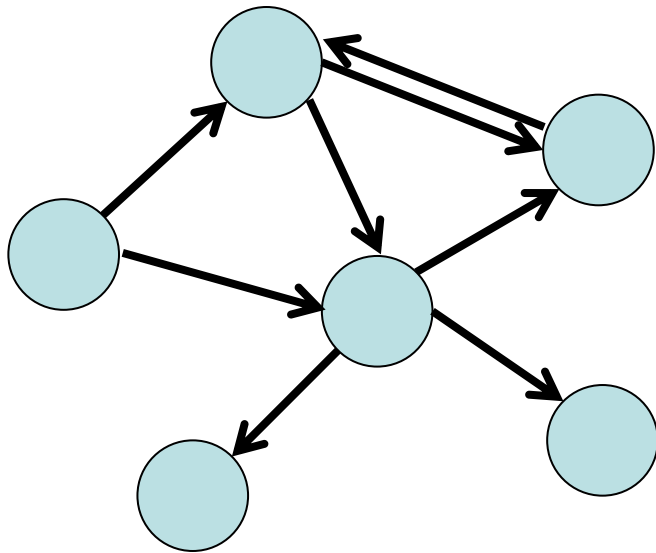
weighted



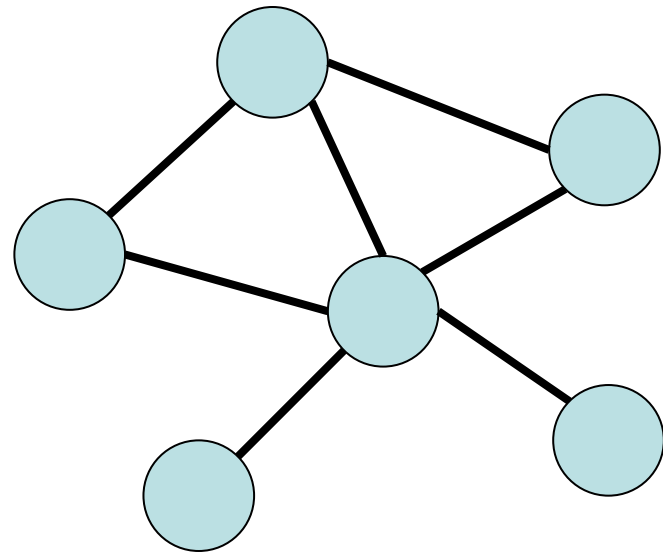
unweighted

Graph Properties

A graph is **directed** if its edges have direction, or **undirected** if its edges do not have direction (are bidirectional).



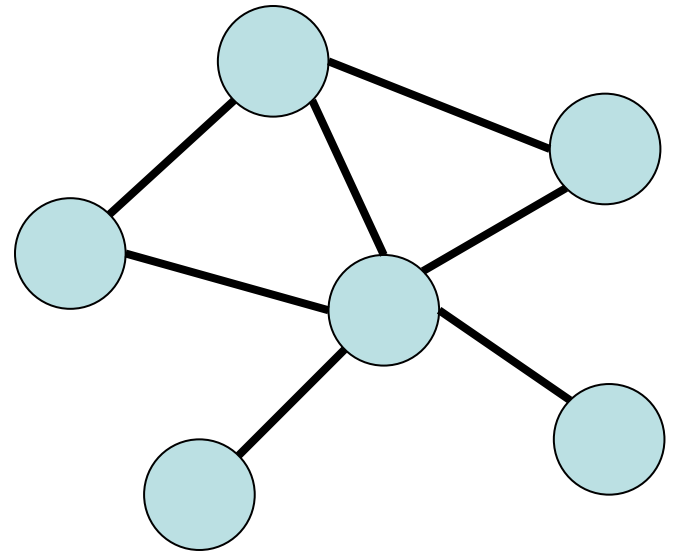
directed



undirected

Graph Properties

- Connected or unconnected
- Acyclic
- Directed or undirected
- Weighted or unweighted
- Complete

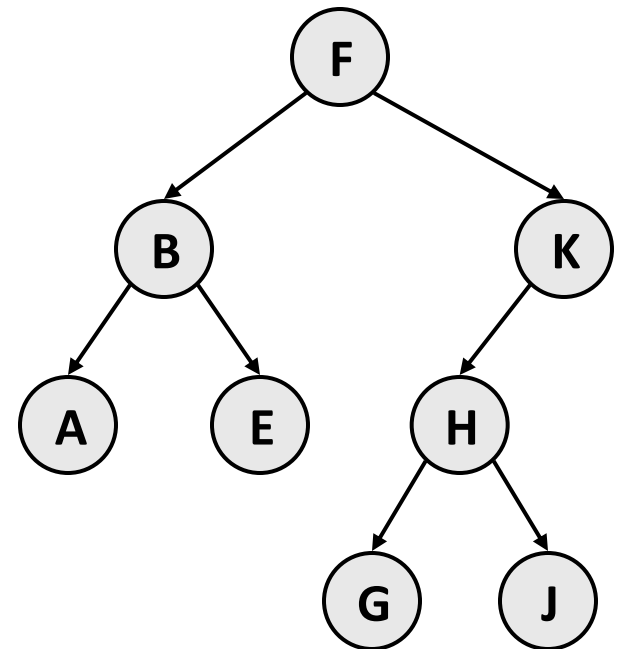
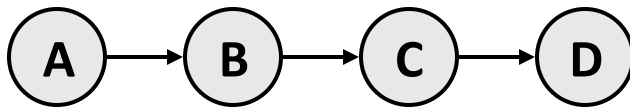


Which of these properties do **binary trees** have?

What about **linked lists**?

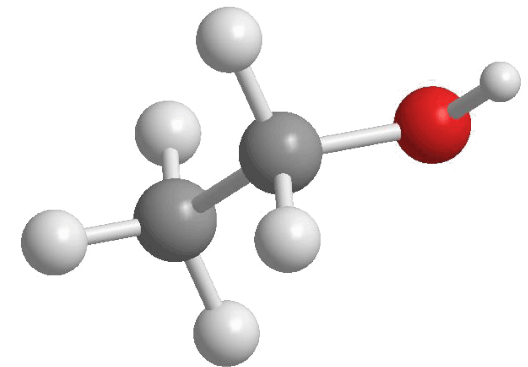
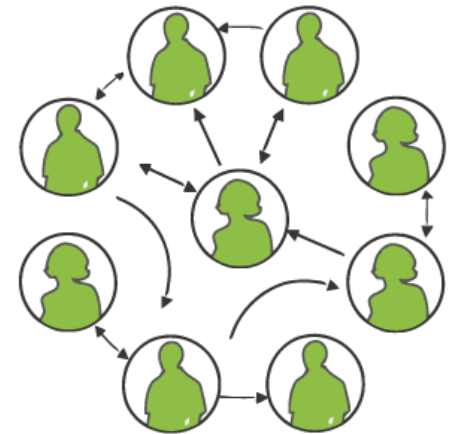
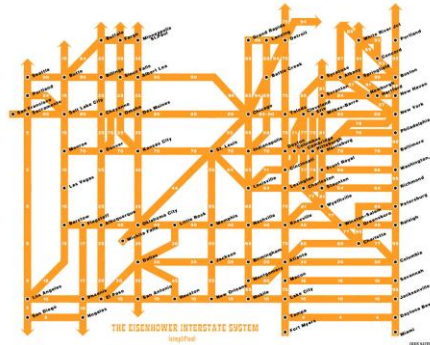
One Big, Happy Family

- A *binary tree* is a graph with some restrictions:
 - The tree is an unweighted, directed, acyclic graph (DAG).
 - Each node's in-degree is at most 1, and out-degree is at most 2.
 - There is exactly one path from the root to every node.
- A *linked list* is also a graph:
 - Unweighted DAG.
 - In/out degree of at most 1 for all nodes.



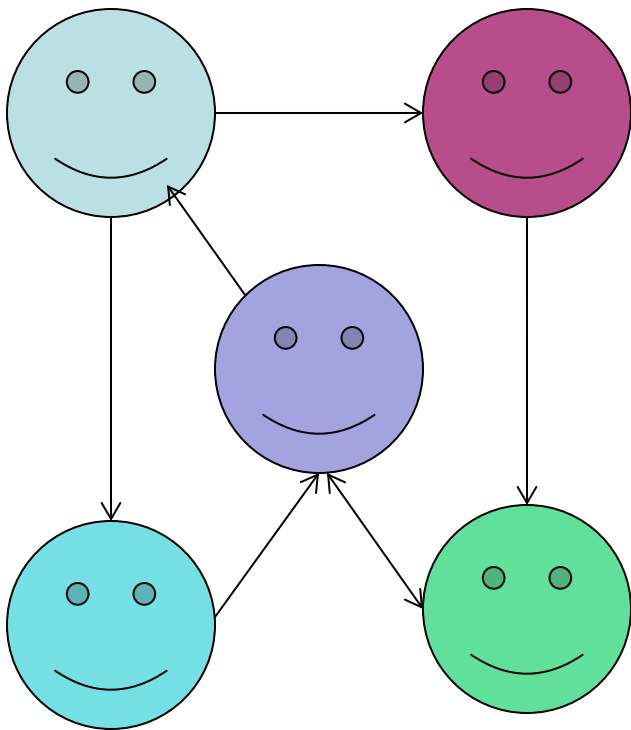
Graph examples

- For each, what are the vertices and what are the edges?
 - Web pages with links
 - Functions in a program that call each other
 - Road maps (e.g., Google maps)
 - Airline routes
 - Facebook friends
 - Course pre-requisites
 - Family trees
 - Paths through a maze
 - Chemical bonds



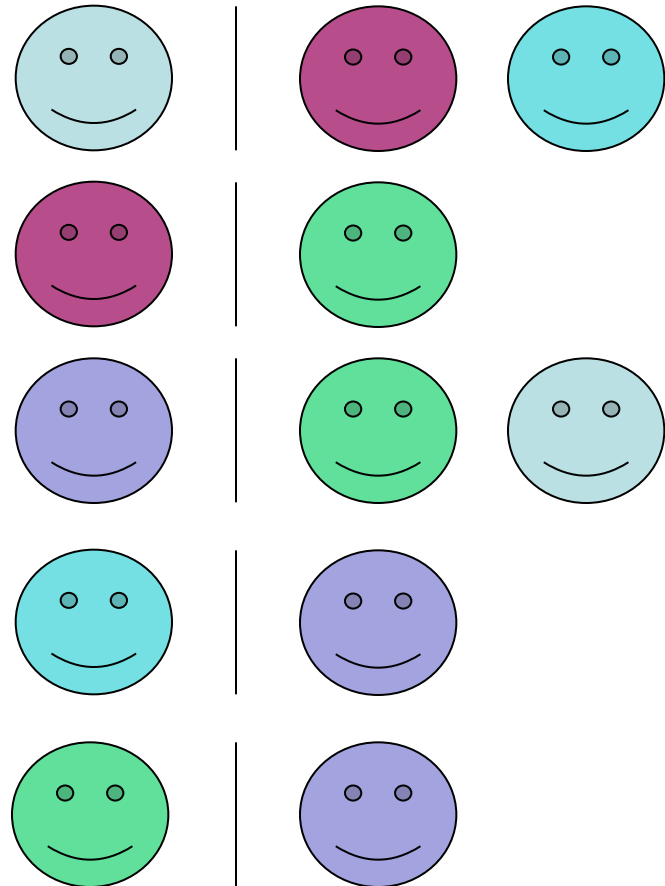
Adjacency List

- Map<**Node**, Vector<**Node**>>
– or Map<**Node**, Set<**Node**>>



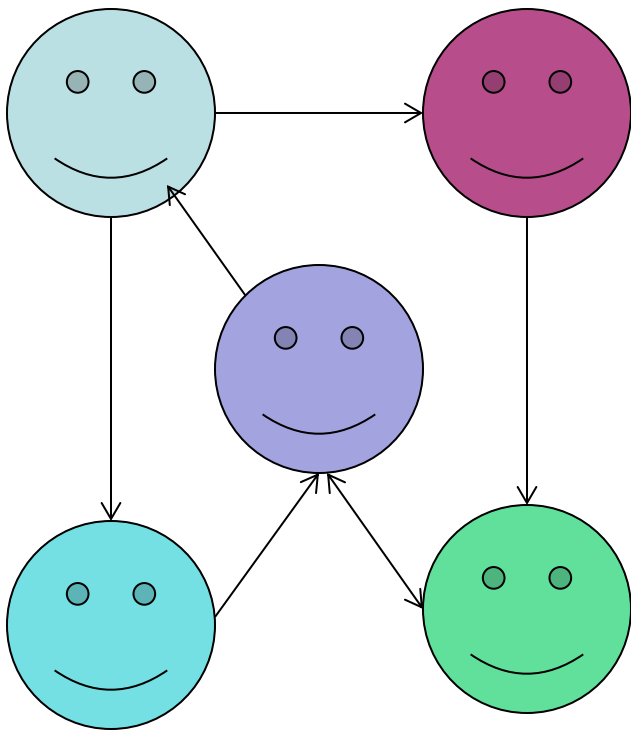
Node

Set<**Node**>



Adjacency Matrix

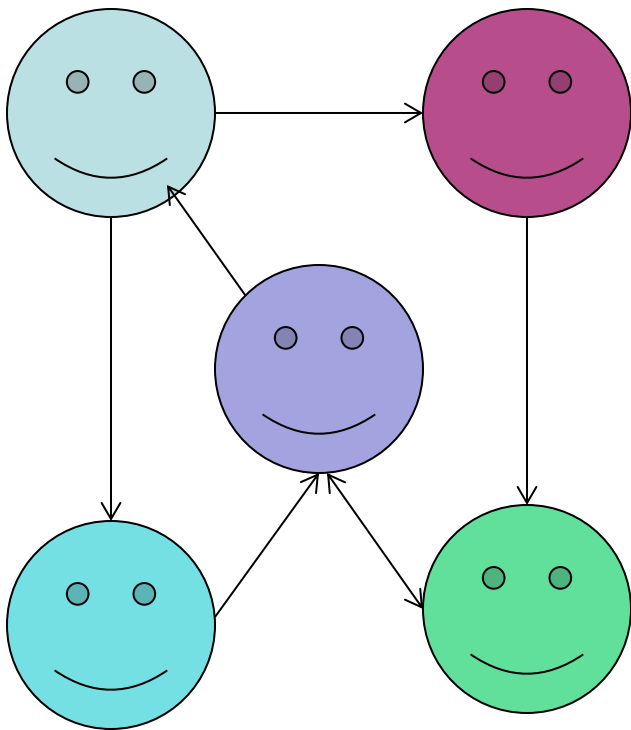
- Store a boolean grid, rows/columns correspond to nodes
 - Alternative to Adjacency List



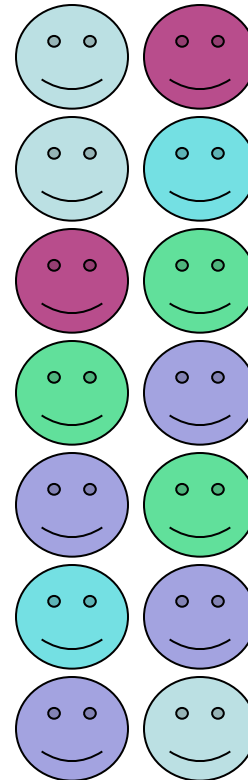
					
	F	T	F	T	F
	F	F	F	F	T
	T	F	F	F	T
	F	F	T	F	F
	F	F	T	F	F

Edge List

- Store a $\text{Vector}\langle \textit{Edge} \rangle$ (or $\text{Set}\langle \textit{Edge} \rangle$)
 - *Edge* struct would have the two nodes



$\text{Vector}\langle \textit{Edge} \rangle$



Twitter Influence



Twitter Influence

- Twitter (now X) lets a user follow another user to see their posts.
- Following is directional (e.g. I can follow you but you don't have to follow me back 😞)
- Let's define being *influential* as having a high number of *followers-of-followers*.
 - Reasoning: doesn't just matter how many people follow you, but whether the people who follow you reach a large audience.
- We can write a function **mostInfluential** that reads a file of Twitter relationships and outputs the most influential user.

